Advance Java LAB

Rajasthan Technical University, Kota (Raj.)

Software Requirements Specification (SRS)

Project Name: PatentPulse

Research Document Management, File Verification
and Internal Review Portal

Submitted By

Mohit Sharma — 24EARAD104

Meet — 24EARAD101

Jay — 24EARAD068

Nitesh — 24EARAD114

Mann Jain — 24EARAD092

Under the supervision of
Er. Ram Babu Buri
Assistant Professor

Session: 2026–27

Department of Artificial Intelligence & Data Science
ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

Table of Contents

Section	Core topics
1. Introduction	Purpose, scope, objectives, conventions, audience and references
2. Overall Description	Perspective, features, user classes, environment, constraints and assumptions
3. Internal Interface Requirements	Input, processing, output, integration and internal quality
4. External Interface Requirements	User, software, hardware and communication interfaces
5. Other Functional Requirements	Traceable functional requirements and acceptance criteria
6. Other Non-Functional Requirements	Security, reliability, performance, usability and maintainability
7. Analysis Models	Use cases, module architecture, workflows and data model
8. Other Requirements	Persistence, privacy, accessibility, deployment, backup and documentation

Document version: 1.0 | **Date:** 13 October 2026

This specification describes the supplied PatentPulse implementation. Operational recommendations are identified separately from implemented application behaviour. It does not claim production certification or measured performance results.

1. Introduction

1.1 Purpose of the System

PatentPulse provides an internal workspace for research drafts, submitted files, version history and review decisions. It connects each decision to an explicit document version and allows authorised users to compare a candidate file against its recorded SHA-256 fingerprint.

1.2 Project Scope

The application supports registration, login, owner-scoped research records, validated uploads, numbered versions, reviewer assignments, decisions, feedback, controlled downloads, audit events and printable receipts. Java 17, Servlets, JSP, JDBC and MySQL form the application stack. The local demonstration uses H2 and embedded Tomcat.

PatentPulse does not perform plagiarism detection, semantic similarity analysis, patentability examination or patent filing. It does not implement blockchain, digital signatures, independent timestamping or malware scanning. A hash match means that candidate bytes match the recorded fingerprint under the trusted-server model.

1.3 Objectives

Preserve version-specific metadata and feedback; control access to research files; make review status understandable; detect changed candidate files; and provide a maintainable Advanced Java project with five clearly assigned modules.

1.4 Document Conventions

FR identifies a functional requirement; NFR identifies a quality requirement. “Shall” describes required application behaviour. Operational recommendations describe deployment work. Draft is an inferred condition of a research item with no submitted version. It is not a document_versions status value.

1.5 Intended Audience

Audience	Recommended sections
Mentor / evaluator	Scope, functional requirements, analysis models and validation boundaries
Development team	Interfaces, requirement matrix, data model and module ownership
Administrator	Access rules, deployment, backup and audit requirements

1.6 References

PatentPulse source: src/main/java/com/patentpulse/, src/main/webapp/, pom.xml, SQL schemas and supplied test reports. Repository: https://github.com/mohitsharma1305/PatentPulse. The supplied JavaLabPortal SRS is used for document organisation only; its application features and deployment claims are not adopted.

2. Overall Description

2.1 Product Perspective

PatentPulse is a browser-based application packaged as a WAR. JSP views render HTML, servlets handle requests, services enforce rules, and the Database helper executes prepared JDBC operations through HikariCP. Uploaded file bytes reside separately from database metadata.

2.2 Product Features and Module Ownership

Member	Module
Mohit Sharma	Drafts, upload validation, numbered submissions, history and coordination
Meet	Registration, login, sessions, account checks and CSRF protection
Jay	SHA-256 generation, candidate comparison and authorised downloads
Nitesh	Assignments, reviewer queue, decisions, feedback and review policy
Mann Jain	User administration, audit display and submission receipts

2.3 User Classes

Visitors register or log in. Authenticated users maintain their own research records; current personal research routes are not restricted to the researcher role alone. Assigned reviewers view and decide on assigned versions. Administrators assign reviewers, manage eligible non-admin accounts and inspect audit events.

2.4 Operating Environment

JDK 17, Maven, a Servlet 4.0 compatible Tomcat 9 runtime, MySQL and a writable private upload directory are required for the external configuration. A modern browser acts as the client. H2 and embedded Tomcat support local demonstration and automated workflows.

2.5 Design Constraints

Use javax.servlet APIs rather than mixing Jakarta namespace dependencies. Store passwords as BCrypt hashes. Keep file bytes outside the public web root. Limit accepted uploads to 10 MiB. Each version accepts at most two distinct reviewers. Filesystem writes and database transactions are not one atomic operation.

2.6 Assumptions and Dependencies

The host and database operator are trusted. Deployment credentials, directory permissions, TLS and backups are configured by the operator. Reviewers must be active and eligible. Internet access may be needed to obtain Maven dependencies. No public hosting service is assumed.

3. Internal Interface Requirements

3.1 Input Interface

Authentication inputs include name, email and password as applicable. Research inputs include title, field, summary and optional existing research identifier. Submission requests include a multipart file. Review inputs include version identifier, reviewer identifier or decision and feedback. State-changing requests carry a session-linked CSRF token.

3.2 Processing Interface

Component	Responsibility
SecurityFilter	Check authentication, active account and POST CSRF token
AuthServlet / AuthService	Validate registration and credentials; establish or invalidate session
AppServlet / LedgerService	Route requests, enforce record permissions and transact state changes
UploadStore / Hashing	Validate and store files; compute streaming SHA-256
ReviewPolicy	Calculate review status and identify terminal states
FileServlet	Authorise version access and stream the stored file
Database	Manage prepared queries, connection pooling, transactions and audit writes

3.3 Output Interface

The system renders authentication, dashboard, editor, document, review queue, user-management, audit and receipt pages. Verification displays MATCH or MISMATCH for the candidate file. Downloads return authorised file bytes. Failed validation must leave the relevant business operation incomplete and provide an error response.

3.4 Integration Interface

UploadStore returns stored-file metadata to the submission service. LedgerService commits a new version and audit event in a database transaction. If persistence fails, the controller attempts to remove the newly stored file. Verification compares a candidate stream to database metadata. FileServlet does not automatically rehash a file during download.

3.5 Non-Functional Internal Requirements

Bind SQL parameters, release JDBC resources, avoid request-specific mutable servlet state, enforce permissions inside service operations and keep secrets out of responses. Database failures must roll back transactional changes. A crash between file storage and database commit can leave an orphan file and requires operational recovery.

4. External Interface Requirements

4.1 User Interface

View	Expected interaction
auth.jsp	Registration and login with validation feedback
dashboard.jsp / editor.jsp	List and search own records; enter draft and submission metadata
document.jsp	View version, history, digest, feedback, assignments and verification
queue.jsp	List reviewer assignments and open their versions
users.jsp / audit.jsp	Manage eligible accounts and inspect recent application events
receipt.jsp	View and print the selected version receipt

4.2 Software Interface

The application uses Java 17, Servlet 4.0 / javax.servlet, JSP, JDBC, MySQL Connector/J, HikariCP and BCrypt as declared by the project dependencies. Maven builds and tests the WAR. H2 and embedded Tomcat are demonstration/test components; they do not replace validation of an external MySQL deployment.

4.3 Hardware Interface

A general-purpose laptop or server capable of running JDK 17, Tomcat and MySQL is sufficient for a development trial. Storage capacity must account for all retained versions and backups. No measured minimum RAM, CPU capacity or concurrent-user limit is established by the supplied evidence.

4.4 Communication Interface

Browsers submit form-encoded and multipart HTTP requests and receive HTML or file streams. The local demonstration uses http://localhost:8080/patentpulse. HTTPS is an operational requirement for deployment beyond a trusted local environment. Database communication uses JDBC; private database credentials must not be sent to the browser.

4.5 Upload Contract

Accepted extensions are pdf, txt, md, java, py, js, c, cpp, h, json and csv. Files must be nonempty and no larger than 10 MiB (10,485,760 bytes). Generated storage names prevent the original filename from selecting the filesystem path. Basic format checks do not constitute antivirus scanning.

5. Other Functional Requirements

ID / owner	Requirement and acceptance check
01 / Meet	The system shall register valid accounts and store BCrypt password hashes. Check valid registration, duplicate email and rejected invalid input.
02 / Meet	The system shall authenticate active accounts, rotate the session identifier at login and invalidate the session on logout. Check that a logged-out session cannot use protected routes.
03 / Meet	The system shall validate CSRF tokens on protected POST requests. Missing or incorrect tokens shall reject state-changing operations.
04 / Mohit	The system shall save owner-scoped draft metadata and list/search the owner’s records. A different user shall not edit that draft by guessing its identifier.
05 / Mohit	The system shall reject empty, oversized or unsupported uploads and store valid uploads under generated names outside the web root.
06 / Mohit	The system shall allocate a new version number and preserve submitted metadata snapshots. Submitting a revision shall not overwrite an earlier version.
07 / Jay	The system shall calculate a SHA-256 digest for submitted bytes and store it with version metadata. Known digest fixtures shall produce expected values.
08 / Jay	The system shall compare an authorised candidate file with the recorded digest and display MATCH or MISMATCH. Altered sample bytes shall yield MISMATCH.
09 / Jay	The system shall allow downloads only to the owner, an assigned reviewer or an administrator. Unrelated users shall be denied.
10 / Nitesh	The system shall allow an administrator to assign at most two distinct active reviewers, excluding the owner. Duplicate, self and third assignments shall fail.
11 / Nitesh	The system shall show reviewers their assigned queue and accept at most one vote per assignment with associated feedback.
12 / Nitesh	The system shall use NEEDS_REVISION after any rejection and INTERNALLY_REVIEWED after two approvals. Terminal versions shall reject further votes and assignments.
13 / Mann	The system shall manage eligible non-admin account roles and active status. Admin-account edits and changes that invalidate unfinished review assignments shall be rejected.
14 / Mann	The system shall record relevant audit events and show the latest 200 in the administration view. Audit records are ordinary database records.
15 / Mann	The system shall display a printable receipt for an authorised version using stored metadata. The receipt shall not claim patent grant or independently verified ownership.

6. Other Non-Functional Requirements

ID / aspect	Requirement and validation approach
NFR-01 Security	Use BCrypt, session account checks, CSRF validation and per-record authorisation. Validate denied access, invalid tokens and inactive accounts.
NFR-02 Data integrity	Preserve version snapshots and uniqueness constraints. Validate transaction rollback and duplicate-assignment rejection.
NFR-03 Reliability	Attempt file cleanup after failed persistence. Test failure cases; establish crash recovery separately. No availability percentage is claimed.
NFR-04 Performance	Use streaming file operations and pooled database connections. Measure latency at selected file sizes and concurrency before defining an SLA.
NFR-05 Usability	Show version status, actionable form errors and clear verification outcomes. Validate with representative task walkthroughs.
NFR-06 Maintainability	Keep views, request handlers, services and persistence responsibilities separate. Coordinate shared-file changes through review.
NFR-07 Portability	Build a WAR for Java 17 and Servlet 4.0 / Tomcat 9. Check startup and schema compatibility on the target environment.
NFR-08 Testability	Retain hash, review-policy and connected workflow tests. Archive build output and identify the tested source revision.
NFR-09 Privacy	Limit file access by ownership, assignment and administration. Avoid exposing database credentials or server paths.
NFR-10 Scalability	Do not claim safe horizontal scaling without validating shared storage, sessions and database concurrency.

6.1 Verification Boundary

The supplied suite contains four hashing tests, four review-policy tests and three connected workflow tests. The existing passing record is functional evidence, not a load test or security audit. External MySQL/Tomcat operation, TLS, backup restoration and concurrent-request behaviour need deployment-specific checks.

7. Analysis Models

7.1 Use Case Model

Actor	Use cases
Visitor	Register, log in
Authenticated owner	Save draft, submit version, search own records, inspect history
Assigned reviewer	Open queue, read assigned version, record decision and feedback
Administrator	Assign reviewers, manage eligible users, inspect audit events
Authorised version user	View details, verify candidate file, download and view receipt

7.2 Class / Module Architecture

Views call servlet routes through the browser. SecurityFilter gates protected requests. AuthServlet uses AuthService; AppServlet uses LedgerService and UploadStore; FileServlet uses access checks and storage lookup. LedgerService uses Database, ReviewPolicy and Hashing. User represents logged-in identity; other persistence records are represented by maps in the supplied implementation.

7.3 Submission and Verification Workflows

Submission: authenticate and validate token; validate multipart input; store file and compute hash; check ownership; create numbered version in a transaction; record audit event; redirect to details. On persistence failure, roll back and attempt file cleanup.

Verification: authenticate; authorise version; hash candidate stream; compare to recorded digest; record outcome; display MATCH or MISMATCH. This checks bytes rather than research novelty or semantic similarity.

7.4 Review State Model

Current state	Event	Result
SUBMITTED	First reviewer assigned	IN_REVIEW
IN_REVIEW	First approval / second assignment	IN_REVIEW
IN_REVIEW	Second approval	INTERNALLY_REVIEWED
IN_REVIEW	Any rejection	NEEDS_REVISION
Terminal state	Additional vote or assignment	Rejected

A revision creates a separate SUBMITTED version. It does not reopen the previous terminal version.

7.5 Relational Data Model

Table	Purpose and relationship
users	Account, password hash, role and active status
blueprints	Parent research item linked to owner
document_versions	Numbered file and metadata snapshot linked to research item
review_assignments	Version, reviewer and assigning administrator; unique version/reviewer pair
review_votes	Decision and feedback; unique assignment
audit_logs	Actor, action, target and details; polymorphic target reference

One research item has many versions. One version has up to two assignments, enforced by the service. One assignment has zero or one vote. Receipts are generated views and have no separate table.

8. Other Requirements

8.1 Data Residency and Persistence

Store relational records in the configured database and file bytes in the private upload directory. The operator chooses the physical host and storage location. Preserve both stores together when moving the application.

8.2 Privacy and Access Scopes

Research records are owner-scoped; assigned reviewers have version-scoped access; administrators have privileged access. Define retention and deletion procedures before handling institutional records. The prototype does not provide a complete legal-consent or compliance management system.

8.3 Accessibility

Use readable labels, visible focus, keyboard-operable controls and errors associated with fields. Test contrast and navigation before release. Formal WCAG conformance has not been established.

8.4 Deployment Environment

Build with Maven and deploy the WAR on Tomcat 9 with MySQL configuration and private storage permissions. Alternatively, use the supplied START-DEMO scripts for local evaluation. config.example.env documents variables but is not automatically loaded. Docker, Railway and CI hosting are not required components of this specification.

8.5 Backup and Disaster Recovery

Operational recommendation: back up the database and uploaded files as one consistent recovery set, protect credentials, and perform a restoration drill. SQL schema scripts alone are not a backup of submissions. No recovery-time objective has been measured.

8.6 Logging and Monitoring

Application audit records capture selected business events. The operator should monitor runtime errors, storage growth and database connectivity. Audit records are not an immutable ledger. Avoid recording passwords or secret configuration in logs.

8.7 System Dependencies

Maintain dependency versions in pom.xml. Required components include Java 17, Maven, Servlet/JSP support, MySQL JDBC, HikariCP and BCrypt. Test/demo dependencies include JUnit, H2 and embedded Tomcat. Use the project dependency file as the version authority.

8.8 Training Materials and Handover

Provide the root README, five module guides, schema scripts, local demo instructions, sample original/modified files, UML material and test reports. Demonstrate a draft, submission, two-reviewer decision, candidate-file mismatch, authorised download and receipt during the viva.

8.9 Shared-File Coordination

LedgerService.java, AppServlet.java and document.jsp are shared integration points. Module owners coordinate changes through small commits and review. UploadStore.java is shared between Mohit’s storage work and Jay’s hashing integration; account and audit data connect Meet and Mann’s modules.